



Revision Notes: Two Dimensional Arrays and Related Problems

In this class, we covered concepts related to two-dimensional arrays specifically focusing on problems like searching in a sorted matrix and calculating the sum of all submatrices. Below is a comprehensive summary of the topics discussed during the class.

Search in Row-Wise and Column-Wise Sorted Matrix

Concept:

- We discussed an efficient way to search for an element in a row-wise and column-wise sorted matrix.
- The approach utilizes the properties of the sorted matrix to minimize the search space by starting from either the top-right or bottom-left corner.

Algorithm:

1. Start from Top-Right or Bottom-Left:

- If starting from the top-right corner:
 - Move left if the element is larger than the target.
 - Move down if the element is smaller than the target.
- Continue this process until the element is found or the matrix boundaries are exceeded.

2. Time Complexity Analysis:

- The time complexity of this approach is $O(M + N)$, where M is the number of rows and N is the number of columns, as in every step, at least one row or one column is discarded from consideration
【6:12+source】 .

Example Scenario:



that column else eliminate the row. This ensures an $O(M + N)$ complexity `【6:5+source】` .

Sum of All Submatrices

Problem Description:

- Calculate the sum of all possible submatrices within a given matrix.

Brute Force Approach:

- Consider all possible submatrices defined by their top-left and bottom-right corners.
- Calculate the sum by iterating over each sub-matrix.

Optimized Approach using Prefix Sums:

1. Brute Force Complexity:

- Enumerating all submatrices leads to a time complexity of $O(N^4)$ for a matrix of size $N \times N$ `【6:16+source】` `【6:19+source】` .

2. Optimized Strategy:

- Use prefix sums to calculate the sum of submatrices in constant time.
- The optimized approach reduces the time complexity to $O(N^3)$ for calculating the sums of all possible submatrices `【6:19+source】` .

Example Illustration:

- Precompute row-wise or column-wise prefix sums. For any submatrix, you can calculate its sum using these prefix sums, thus avoiding recalculating sums from scratch for overlapping submatrices.

Submatrix Contribution:

- Calculate how many submatrices a particular cell contributes to by counting all possible top-left and bottom-right corners the cell can be part of `【6:16+source】` `【6:18+source】` .



-
- The class concluded with a discussion on the application of prefix sums in optimizing matrix operations and the concept of submatrix counting for efficient computation of matrix properties.
 - Students were engaged with problem-solving strategies to cement their understanding of the topics discussed in class.

This class focused on efficiently handling typical operations in two-dimensional arrays, equipping learners with techniques vital in solving comprehensive problems in software development contexts.